

KẾ HOẠCH CHI TIẾT THIẾT KẾ & PHÁT TRIỂN ỨNG DỤNG ĐA NỀN TẢNG (OFFLINE-FIRST & ĐỒNG BỘ P2P KIỂU ZALO)

tài liệu kiến trúc kỹ thuật và lộ trình triển khai chi tiết cho hệ thống ứng dụng đa nền tảng offline-first với cơ chế truyền dữ liệu ngang hàng (P2P).

1. TỔNG QUAN DỰ ÁN & MỤC TIÊU CỐT LÕI

Mục tiêu

Chuyển đổi Web App hiện hữu thành ứng dụng cài đặt đa nền tảng macOS và iOS với khả năng hoạt động độc lập và đồng bộ linh hoạt.

Mô hình Offline-First

Dữ liệu và hình ảnh lưu trữ trực tiếp trên bộ nhớ cục bộ của thiết bị bằng SQLite và Local File Storage. Đảm bảo ứng dụng chạy mượt mà 100% khi mất kết nối mạng. Tất cả các thao tác đọc/ghi đều ưu tiên thực hiện tại CSDL nội bộ trước khi đẩy vào hàng chờ đồng bộ.

Yêu cầu bộ nhớ RAM & Tối ưu tài nguyên cục bộ

Để đảm bảo ứng dụng vận hành mượt mà 100% trong điều kiện offline, thiết bị cần đáp ứng dung lượng RAM tối thiểu từ 2GB (trên iOS) và 4GB (trên macOS). Việc tối ưu bộ nhớ RAM giúp SQLite xử lý mượt mà các truy vấn phức tạp, bộ nhớ đệm (cache) dữ liệu lớn và chia nhỏ luồng truyền tải hình ảnh P2P trực tiếp trên client mà không làm gián đoạn trải nghiệm người dùng.

Cơ chế đồng bộ dữ liệu chữ (Text)

Đồng bộ hai chiều (Bi-directional) giữa App và VPS qua API Push/Pull vi sai (Delta Sync). Khi có đơn hàng mới được tạo từ thiết bị di động đẩy lên VPS, hệ thống sẽ kích hoạt **Webhook** sang **n8n** để gửi thông báo tức thì qua **Bot Telegram**.

Cơ chế đồng bộ hình ảnh kiểu Zalo (P2P/Direct Stream)

- **Tối ưu chi phí lưu trữ:** Ảnh lưu trực tiếp trên bộ nhớ máy (không lưu vĩnh viễn trên VPS để tránh phình dung lượng lưu trữ hạ tầng server).
- **Kênh báo hiệu (Signaling):** Khi thiết bị mới (Máy tính) cần đồng bộ hình ảnh từ thiết bị cũ (Điện thoại), VPS đóng vai trò môi giới kết nối (Signaling Server).

- **Truyền tải trực tiếp:** Ảnh được truyền trực tiếp (P2P qua WebRTC Data Channel hoặc WebSocket Stream trong mạng LAN/Internet).
- **Trải nghiệm người dùng:** Ứng dụng hiển thị thông báo trạng thái: "Đang đồng bộ hình ảnh... Vui lòng không đóng ứng dụng trên điện thoại và giữ kết nối mạng".

2. KIẾN TRÚC HỆ THỐNG & CÔNG NGHỆ (TECH STACK)

Phân lớp hệ thống	Công nghệ lựa chọn	Vai trò & Chức năng
Giao diện đa nền tảng	React, Vite, Tailwind CSS	Xây dựng UI/UX đồng nhất, tối ưu hiệu năng render và dung lượng bundle.
Đóng gói di động	Capacitor (@capacitor/core, @capacitor/cli, @capacitor/filesystem)	Biên dịch sang iOS (IPA), cung cấp Native API truy cập bộ nhớ.
Đóng gói máy tính	Tauri (Rust-based) hoặc Capacitor Desktop	Biên dịch cho macOS (.app/.dmg). Tối ưu dung lượng installation file (< 15MB) và tiêu thụ RAM cực thấp.
Cơ sở dữ liệu cục bộ	SQLite (@capacitor-community/sqlite trên Mobile, better-sqlite3 trên Desktop)	Lưu trữ dữ liệu cấu trúc local, hỗ trợ truy vấn ACID offline đầy đủ. Tối ưu bộ nhớ cache trên RAM để truy vấn dữ liệu lớn mượt mà.
Truyền dẫn P2P & Đồng bộ ảnh	WebRTC (DataChannel) & WebSocket (Socket.io)	Kết nối P2P trực tiếp giữa hai thiết bị. WebSocket Relay đóng vai trò fallback khi NAT traversal thất bại.
Hạ tầng VPS & Tự động hóa	Nginx, Node.js (Express) / FastAPI, PostgreSQL, n8n, Telegram API	Quản lý API đồng bộ text, Signaling Server, xử lý Webhook và gửi thông báo tự động.

3. THIẾT KẾ CƠ SỞ DỮ LIỆU SQLITE CỤC BỘ (SCHEMA)

Dưới đây là cấu trúc các bảng trong cơ sở dữ liệu SQLite tại mỗi thiết bị (iOS & macOS):

Bảng customers

- `id`: TEXT PRIMARY KEY (UUID v4)
- `customer_code`: TEXT UNIQUE NOT NULL
- `name`: TEXT NOT NULL
- `phone`: TEXT
- `address`: TEXT
- `sync_status`: INTEGER DEFAULT 0 (0: Chưa đồng bộ, 1: Đã đồng bộ)
- `updated_at`: INTEGER NOT NULL (Timestamp ms)
- `is_deleted`: INTEGER DEFAULT 0 (0: Hoạt động, 1: Đã xóa mềm)

Bảng products

- `id`: TEXT PRIMARY KEY (UUID v4)
- `product_code`: TEXT UNIQUE NOT NULL
- `name`: TEXT NOT NULL
- `unit`: TEXT
- `base_price`: REAL DEFAULT 0
- `local_image_path`: TEXT
- `sync_status`: INTEGER DEFAULT 0
- `updated_at`: INTEGER NOT NULL
- `is_deleted`: INTEGER DEFAULT 0

Bảng orders

- `id`: TEXT PRIMARY KEY (UUID v4)
- `order_code`: TEXT UNIQUE NOT NULL (Tiền tố: MOB-XXXX hoặc MAC-XXXX)
- `customer_id`: TEXT NOT NULL
- `total_amount`: REAL DEFAULT 0
- `note`: TEXT
- `is_printed`: INTEGER DEFAULT 0 (0: Chưa in, 1: Đã in)
- `local_image_path`: TEXT
- `image_sync_status`: INTEGER DEFAULT 0 (0: Chưa có ảnh/Chưa đồng bộ, 1: Đã đồng bộ)
- `sync_status`: INTEGER DEFAULT 0
- `created_at`: INTEGER NOT NULL
- `updated_at`: INTEGER NOT NULL

- `is_deleted: INTEGER DEFAULT 0`

Bảng `order_items`

- `id: TEXT PRIMARY KEY (UUID v4)`
- `order_id: TEXT NOT NULL`
- `product_id: TEXT NOT NULL`
- `quantity: REAL NOT NULL`
- `unit_price: REAL NOT NULL`
- `amount: REAL NOT NULL`
- `sync_status: INTEGER DEFAULT 0`
- `updated_at: INTEGER NOT NULL`

Bảng `app_metadata`

- `key_name: TEXT PRIMARY KEY`
- `key_value: TEXT NOT NULL` (Lưu các thông số: `last_sync_time`, `device_id`, `active_user_id`)

4. CHI TIẾT CÁC CƠ CHẾ ĐỒNG BỘ

A. Đồng bộ dữ liệu Text (Push / Pull vi sai)

Quy trình Push (Đẩy dữ liệu mới/sửa từ Local lên VPS)

1. Device quét SQLite tìm các bản ghi có `sync_status = 0`.
2. Đóng gói payload gửi đến `POST /api/sync/push`.
3. VPS nhận dữ liệu, ghi vào PostgreSQL trong một DB Transaction.
4. VPS trả về danh sách các `id` đã lưu thành công. Device cập nhật `sync_status = 1` cho các `id` đó.
5. VPS gửi sự kiện Webhook tới **n8n Workflow** nếu có đơn hàng mới (`orders`) để format tin nhắn và push tới Telegram Bot của quản lý.

Quy trình Pull (Kéo dữ liệu thay đổi từ VPS về Local)

1. Device lấy `last_sync_time` từ bảng `app_metadata`.
2. Gọi API `GET /api/sync/pull?since={last_sync_time}`.
3. VPS truy vấn PostgreSQL lấy toàn bộ bản ghi có `updated_at > since`.
4. Device nhận dữ liệu, thực hiện Upsert (Chèn mới hoặc Cập nhật) vào SQLite local, sau đó cập nhật lại `last_sync_time`.

Giải quyết xung đột (Conflict Resolution)

- **Cơ chế:** Áp dụng **Last-Write-Wins (LWW)** dựa trên trường timestamp `updated_at`.
- **Định danh:** Tất cả khóa chính dùng **UUID v4** khởi tạo tại client, triệt tiêu hoàn toàn khả năng trùng ID khi tạo mới dữ liệu ở các thiết bị khác nhau trong trạng thái offline.

B. Đồng bộ hình ảnh trực tiếp kiểu Zalo (Phone → PC)



1. Khởi tạo kết nối:

Khi máy tính Mac mở ứng dụng hoặc người dùng kích hoạt "*Đồng bộ từ điện thoại*", máy tính Mac truy vấn SQLite chọn các đơn hàng có

`local_image_path` IS NULL hoặc `image_sync_status` = 0. Máy tính Mac gửi thông điệp yêu cầu Sync kèm danh sách `order_id` tới VPS qua WebSocket.

2. **Xác thực & Báo hiệu (Signaling):**

VPS định tuyến yêu cầu tới thiết bị Điện thoại thuộc cùng tài khoản/cửa hàng. Điện thoại hiển thị thông báo UI: "Đang đồng bộ hình ảnh sang máy tính. Vui lòng giữ ứng dụng mở".

3. **Truyền tải luồng dữ liệu (Data Streaming):**

- Điện thoại mở file ảnh từ bộ nhớ nội bộ (@`capacitor/filesystem`), chia nhỏ thành các gói nhị phân (Binary Chunks 64KB).
- Gửi trực tiếp qua **WebRTC DataChannel**. Nếu kết nối P2P thất bại do Firewall/Symmetric NAT, hệ thống tự động fallback sang truyền qua WebSocket Relay của VPS.

4. **Hoàn tất & Cập nhật trạng thái:**

Máy tính Mac nhận đủ các chunks, nối lại thành file ảnh hoàn chỉnh và lưu vào thư mục ứng dụng trên ổ cứng. Cập nhật `local_image_path` và `image_sync_status` = 1 trong SQLite local.

VPS hoàn toàn không lưu trữ file ảnh, tối ưu 100% dung lượng đĩa cứng server.

1. 5. TÍCH HỢP QUY TRÌNH IN ĐƠN TẠI MÁY TÍNH MAC & THIẾT BỊ iOS

2. **Nhận đơn:** Nhân viên mở app trên máy tính Mac hoặc iPhone/iPad, dữ liệu đơn hàng mới tự động cập nhật về thiết bị qua Delta Pull.
3. **Đồng bộ ảnh đính kèm:** Hình ảnh chứng từ/sản phẩm được truyền P2P giữa các thiết bị iOS và macOS.
4. **Cấu hình mẫu in:** Tích hợp bộ thiết kế mẫu in hỗ trợ các khổ giấy:
 - Giấy in nhiệt: K80 (80mm), K57 (57mm).
 - Giấy văn phòng: A4, A5 (In phiếu xuất kho, hóa đơn bán hàng).
5. **Thực thi lệnh in (Native Printing):**
 - Ứng dụng Tauri/Capacitor gọi API giao tiếp trực tiếp với Native Print Driver (CUPS) của macOS hoặc AirPrint trên iOS.

6. LỘ TRÌNH TRIỂN KHAI THEO TỪNG GIAI ĐOẠN (ROADMAP)

Giai đoạn 1: Hạ tầng VPS Backend, Nginx & n8n

- **Phân vùng VPS độc lập (Namespace/Partitioning):** Thiết lập môi trường isolated riêng (Database schema/Namespace độc lập, API endpoint phụ như `/api/v2/` hoặc subdomain riêng `dev-sync.domain.com`) dành riêng cho việc

thử nghiệm cơ chế đồng bộ mới. Tách biệt hoàn toàn với Web App hiện tại để đảm bảo an toàn dữ liệu và tránh ảnh hưởng đến hệ thống đang vận hành.

- Khởi tạo VPS, cấu hình Nginx Reverse Proxy và chứng chỉ SSL Let's Encrypt.
- Xây dựng CSDL PostgreSQL trung tâm.
- Viết các API cơ bản và cài đặt môi trường n8n kết nối Telegram Bot.

Giai đoạn 2: Lớp lưu trữ cục bộ SQLite & File Management

- Tích hợp thư viện SQLite cho React (Capacitor SQLite & Better-SQLite3).
- Thực thi scripts tạo bảng (`customers`, `products`, `orders`, `order_items`, `app_metadata`).
- Xây dựng Service đọc/ghi file ảnh vào bộ nhớ cục bộ thiết bị.

Giai đoạn 3: Sync Engine dữ liệu Text

- Viết module tính toán sai biệt dữ liệu (Delta Calculation) dựa trên `updated_at`.
- Triển khai logic `POST /api/sync/push` và `GET /api/sync/pull`.
- Tích hợp xử lý xung đột dữ liệu theo cơ chế LWW.

Giai đoạn 4: Kênh đồng bộ ảnh P2P kiểu Zalo

- Xây dựng Signaling Server bằng Socket.io trên Node.js Backend.
- Tích hợp WebRTC DataChannel trên Client (iOS & macOS).
- Xây dựng UI thông báo trạng thái đồng bộ và cơ chế chia nhỏ/ghép gói nhị phân (Binary Chunking).

Giai đoạn 5: Đóng gói đa nền tảng & Module in ấn

- Cấu hình Capacitor đóng gói ứng dụng iOS (`.ipa`).
- Cấu hình Tauri/Capacitor đóng gói ứng dụng macOS (`.app` / `.dmg`).
- Phát triển Module in ấn kết nối máy in nhiệt K80 và máy in A4/A5 qua macOS Native Driver (CUPS) và iOS AirPrint.
- Tích hợp cơ chế tự động cập nhật phần mềm (Auto-Update): Cấu hình Sparkle Framework hoặc Tauri Updater cho bản macOS, và thiết lập luồng cập nhật OTA/TestFlight/App Store cho bản iOS.

Giai đoạn 6: Kiểm thử thực tế & Bàn giao

- Kiểm thử kịch bản ngắt mạng 100%, tạo đơn hàng offline trên điện thoại.
- Mở mạng, kiểm tra tiến trình Push dữ liệu text lên VPS và trigger Telegram Bot.
- Kiểm thử đồng bộ ảnh P2P trực tiếp từ iPhone sang Mac và thực hiện silent print đơn hàng.